

USB Handle Control (Optional)

Install the Handle



Insert the USB cable of the wired gamepad into a USB port on the Windows computer. For a wireless gamepad, insert the USB receiver into the computer and make sure it is recognized by the virtual machine.

Run the Handle Control Program

Code path:

```
/home/yahboom/Dofbot/1.telecontrol/4.arm_handle/arm_handle_old.ipynb
```

The following code steps must be executed one at a time based on actual conditions; do not run all of them at once.

```
# Gamepad Remote Control  
In this example, we will use a gamepad controller connected to a web browser to  
remotely control the robot arm.
```

Create a Gamepad Controller

The first thing we need to do is create an instance of the 'Controller' widget, which we will use to drive our Dofbot-se.

The "Controller" widget accepts an "index" parameter that specifies the controller number. If you want to correctly use your gamepad to control the robot arm, you can:

1. Open <http://html5gamepad.com> in your browser.
2. Press a button on the gamepad you are using
3. Remember the corresponding index number that appears after pressing the button

Next, we will use that index to create and display the controller.

```
import ipywidgets.widgets as widgets

controller = widgets.Controller(index=0) # Replace with the index number of the
controller you just tested and are using

display(controller)
```

```
# Library path imports
import threading
import time

# Thread operation library
import inspect
import ctypes

# Import the robot arm object
from Arm_Lib import Arm_Device
Arm = Arm_Device()
```

```
#Create a method to actively stop the process

def _async_raise(tid, exctype):

    """raises the exception, performs cleanup if needed"""

    tid = ctypes.c_long(tid)

    if not inspect.isclass(exctype):
        exctype = type(exctype)

    res = ctypes.pythonapi.PyThreadState_SetAsyncExc(tid,
        ctypes.py_object(exctype))

    if res == 0:

        raise ValueError("invalid thread id")
    elif res != 1:
```

```

        # ""if it returns a number greater than one, you're in trouble,

        # and you should call it again with exc=NULL to revert the effect""

        ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, None)
def stop_thread(thread):

    _async_raise(thread.ident, SystemExit)

```

```

# Create a method for gamepad joystick control of the robot arm

## If the gamepad's analog mode is turned on (red light on), the left direction
keys will not work. Please use the left and right joysticks and the L/R keys on
both sides to control the robot arm.

## Program functions:

1. The left joystick and direction keys control servo 1 and servo 2. Left and
right control servo 1's left and right movement; up and down control servo 2's
forward and backward movement.

2. The right joystick and number keys control servo 5 and servo 6. Left and
right control servo 5's left and right rotation; up and down control servo 6's
gripper tightening and releasing.

3. L1 and L2 control servo 3 moving forward or backward.

4. R1 and R2 control servo 4 moving forward or backward.

5. Pressing the SELECT key sets all robot arm servos to 90 degrees.

```

```

def Arm_Handle():

    s_time = 500

    s_step = 1

    angle_1 = angle_2 = angle_3 = angle_4 = angle_5 = angle_6 = 90

    while 1:

        # Due to individual differences in joysticks, the reset value may not be
        zero, so 0.1 is used as a filter to avoid accidental operation.

        # Servo 2, A1: negative is up, positive is down

        if controller.axes[1].value <= 0.1 and controller.axes[1].value >= -0.1:

            time.sleep(.000001)

        else:

            if controller.axes[1].value > 0.1:

                angle_2 += s_step

```

```

        else:

            angle_2 -= s_step

        if angle_2 > 180:

            angle_2 = 180

        elif angle_2 < 0:

            angle_2 = 0

        Arm.Arm_serial_servo_write(2, angle_2, s_time)

        time.sleep(0.01)

# Servo 1, A0: negative is left, positive is right

if (controller.axes[0].value <= 0.1 and controller.axes[0].value >=
-0.1):

    time.sleep(.000001)

else:

    if controller.axes[0].value > 0.1:

        angle_1 -= s_step

    else:

        angle_1 += s_step

    if angle_1 > 180:

        angle_1 = 180

    elif angle_1 < 0:

        angle_1 = 0

    Arm.Arm_serial_servo_write(1, angle_1, s_time)

    time.sleep(0.01)

# Servo 6, NUM1=B0, NUM3=B2, A2: negative is up, positive is down

if controller.buttons[0].value == True:

    angle_6 += s_step

```

```
        if angle_6 > 180:

            angle_6 = 180

        elif angle_6 < 0:

            angle_6 = 0

        Arm.Arm_serial_servo_write(6, angle_6, s_time)

        time.sleep(0.01)

elif controller.buttons[2].value == True:

    angle_6 -= s_step

    if angle_6 > 180:

        angle_6 = 180

    elif angle_6 < 0:

        angle_6 = 0

    Arm.Arm_serial_servo_write(6, angle_6, s_time)

    time.sleep(0.01)

elif controller.axes[2].value > 0.5:

    angle_6 -= s_step

    if angle_6 > 180:

        angle_6 = 180

    elif angle_6 < 0:

        angle_6 = 0

    Arm.Arm_serial_servo_write(6, angle_6, s_time)

    time.sleep(0.01)

elif controller.axes[2].value < -0.5:

    angle_6 += s_step

    if angle_6 > 180:

        angle_6 = 180

    elif angle_6 < 0:

        angle_6 = 0

    Arm.Arm_serial_servo_write(6, angle_6, s_time)
```

```
time.sleep(0.01)

# Servo 5, NUM2=B1, NUM4=B3, A5: negative is left, positive is right

if controller.buttons[1].value == True:

    angle_5 += s_step

    if angle_5 > 180:

        angle_5 = 180

    elif angle_5 < 0:

        angle_5 = 0

    Arm.Arm_serial_servo_write(5, angle_5, s_time)

    time.sleep(0.01)

elif controller.buttons[3].value == True:

    angle_5 -= s_step

    if angle_5 > 180:

        angle_5 = 180

    elif angle_5 < 0:

        angle_5 = 0

    Arm.Arm_serial_servo_write(5, angle_5, s_time)

    time.sleep(0.01)

elif controller.axes[5].value > 0.5:

    angle_5 += s_step

    if angle_5 > 180:

        angle_5 = 180

    elif angle_5 < 0:

        angle_5 = 0

    Arm.Arm_serial_servo_write(5, angle_5, s_time)

    time.sleep(0.01)

elif controller.axes[5].value < -0.5:

    angle_5 -= s_step
```

```
        if angle_5 > 180:

            angle_5 = 180

        elif angle_5 < 0:

            angle_5 = 0

        Arm.Arm_serial_servo_write(5, angle_5, s_time)

        time.sleep(0.01)

# Servo 4, R1=B5, R2=B7

if controller.buttons[5].value == True:

    angle_4 -= s_step

    if angle_4 > 180:

        angle_4 = 180

    elif angle_4 < 0:

        angle_4 = 0

    Arm.Arm_serial_servo_write(4, angle_4, s_time)

    time.sleep(0.01)

elif controller.buttons[7].value == True:

    angle_4 += s_step

    if angle_4 > 180:

        angle_4 = 180

    elif angle_4 < 0:

        angle_4 = 0

    Arm.Arm_serial_servo_write(4, angle_4, s_time)

    time.sleep(0.01)

# Servo 3, L1=B4, L2=B6

if controller.buttons[4].value == True:

    angle_3 -= s_step

    if angle_3 > 180:

        angle_3 = 180

    elif angle_3 < 0:
```

```

        angle_3 = 0

        Arm.Arm_serial_servo_write(3, angle_3, s_time)

        time.sleep(0.01)

    elif controller.buttons[6].value == True:

        angle_3 += s_step

        if angle_3 > 180:

            angle_3 = 180

        elif angle_3 < 0:

            angle_3 = 0

        Arm.Arm_serial_servo_write(3, angle_3, s_time)

        time.sleep(0.01)

# Press SELECT button B8 to set all robot arm servos to 90 degrees

if controller.buttons[8].value == True:

    angle_1 = angle_2 = angle_3 = angle_4 = angle_5 = angle_6 = 90

    Arm.Arm_serial_servo_write6(90, 90, 90, 90, 90, 90, 1000)

    time.sleep(1)

```

#Run the cell code below to start the gamepad real-time control thread

#After waiting for the gamepad control thread to start, you can control the robot arm with the gamepad.

```
thread2 = threading.Thread(target=Arm_Handle)
```

```
thread2.setDaemon(True)
```

```
thread2.start()
```

#End the gamepad thread; if this cell is run, the gamepad cannot control the robot arm.

#If thread start or end fails,

#please restart the kernel and run from the beginning again.

```
stop_thread(thread2)
```


Handle Control

The left joystick controls servo 1 and servo 2. Moving left controls servo 1 to the left; moving right controls servo 1 to the right. Moving up controls servo 2 forward; moving down controls servo 2 backward.

The left direction keys have the same functions as the left joystick.

The right joystick controls servo 5 and servo 6. Moving left controls servo 5 to rotate left; moving right controls servo 5 to rotate right. Moving up controls servo 6 to tighten the gripper; moving down controls servo 6 to release the gripper.

The right buttons have the same functions as the right joystick.

L1 controls servo 3 forward; L2 controls servo 3 backward.

R1 controls servo 4 forward; R2 controls servo 4 backward.

The SELECT key sets all robot arm servos to 90 degrees, standing vertically upright.

